

Employee Management System (EMS) - Architecture Overview

1. Main Application & Configuration (Setup)

EmsApplication: The main entry point that starts the Spring Boot server. It includes OpenAPI annotations to configure the Swagger UI so it correctly displays the "Authorize" button for JWTs.

ApplicationConfig: Sets up essential Spring Security beans. It tells Spring to use BCrypt for hashing passwords, creates the AuthenticationProvider to link your database users to Spring Security, and exposes the AuthenticationManager used during the login process.

SecurityConfig: The main security rulebook. It disables CSRF, sets the application to stateless (no sessions, just JWTs), makes specific paths public (like /auth/** and Swagger), forces authentication for everything else, and inserts the custom JWT filter into the request pipeline.

2. Security & JWT Processing

JwtAuthenticationFilter: A custom security filter that runs on every incoming HTTP request. It extracts the JWT from the Authorization: Bearer <token> header, verifies its validity, and securely logs the user into the Spring SecurityContext.

JwtService: A utility class responsible for the cryptography behind JWTs. It generates new tokens with a secret key, parses incoming tokens to extract the user's email or role, and checks for token expiration.

UserDetailsServiceImpl: Implements Spring Security's UserDetailsService. It takes an email, finds the corresponding Employee in the database, and wraps their data (and roles) into a UserDetails object that Spring Security can understand.

3. Controllers (API Endpoints)

AuthController: The entry point for public authentication requests. It accepts POST requests at /auth/login and passes the user's credentials to the AuthService.

EmployeeController: The entry point for employee management API endpoints. It handles HTTP requests (GET, POST, PUT, DELETE) to manage employees. It utilizes @PreAuthorize annotations to ensure only users with specific roles (ADMIN or EMPLOYEE) can trigger certain actions.

4. Services (Business Logic)

AuthService (Interface) & AuthServiceImpl (Class): The core logic for authentication. During a login attempt, it asks the

AuthenticationManager to verify the credentials. If correct, it fetches the user from the database, asks JwtService to generate a token, and returns it.

EmployeeService (Interface): Defines the contract for all employee-related business operations, such as adding, updating, deleting, searching, and sorting employees.

5. Repository (Data Access)

EmployeeRepository: A Spring Data JPA interface. It handles all database interactions for the Employee entity. By declaring method names like findByEmail or existsByPhoneNumber, Spring automatically generates and executes the underlying SQL queries.

6. Entities & DTOs (Data Models)

Employee: The main database entity. Using @Entity, Hibernate automatically maps this Java class to the employees table in the MySQL database.

LoginRequest: A Data Transfer Object (DTO) used to securely capture incoming JSON for a login attempt, utilizing validation annotations like @NotBlank and @Email.

AuthResponse: A simple DTO used to wrap the generated JWT string into a clean JSON object to send back to the client after a successful login.

EmployeeRequest: A DTO used for creating or updating employees. It strictly controls exactly what fields a user is allowed to submit, preventing malicious overriding of protected database fields (like id or createdAt).